



ECE PARIS • LYON
ÉCOLE D'INGÉNIEURS

Algorithmique et programmation structurée en C

Cours n°6

Langage C

Mémoire, adresses, pointeurs et passage par adresse

Antoine Hintzy



Plan du cours

1. **Mémoire centrale**
2. **Adresses mémoire**
3. **Pointeurs**
4. **Passage par de paramètres par valeur/adresse**

Mémoire Centrale

Rappel : dans un programme, les variables sont stockées en mémoire centrale (mémoire vive / RAM) afin d'être utilisables par le processeur.

La mémoire centrale est composée de plusieurs mots binaires.

De nos jours, un mot binaire fait **un octet** (8 bits). On dit alors que **l'octet est la plus petite unité de mémoire adressable**.

Chaque mot binaire/octet possède une **adresse**.

Une adresse tous les 8 bits.

Les adresses sont écrites au format **Hexadécimal** (alphabet : **0–9 , a–f**).

Exemple : **ef3c4a68**, noté **0xef3c4a68** (**0x** précise le format **hexadécimal**).

Le format Hexadécimal

Base : 16

Alphabet : 0 , 1 , 2 , 3 , 4 , 5 , 6 , 7 , 8 , 9 , A , B , C , D , E , F .

NB : possibilité d'écrire les lettres en majuscules ou en minuscules.

Comptons en Hexadécimal :

0,	1,	2,	3,	4,	5,	6,	7,	8,	9,	A,	B,	C,	D,	E,	F,
10,	11,	12,	13,	14,	15,	16,	17,	18,	19,	1A,	1B,	1C,	1D,	1E,	1F,
20,	21,	22,	23,	24,	25,	26,	27,	28,	29,	2A,	2B,	2C,	2D,	2E,	2F,
3...	4...	5...	6...	7...	8...										
90,	91,	92,	93,	94,	95,	96,	97,	98,	99,	9A,	9B,	9C,	9D,	9E,	9F,
A0,	A1,	A2,	...									FC,	FD,	FE,	FF,
100,	101,	102,	103,	...											

Le format Hexadécimal

Exemples d'affichage

On peut afficher une adresse au format hexadécimal dans un **printf** en utilisant le *placeholder* **%x**.

```
printf("Affichage d'un entier relatif (%%d) : %d %d\n", 15, 0x15); // Affiche :  
// Affichage d'un entier relatif (%d) : 15 21
```

```
printf("Affichage d'un entier exprimé en hexadécimal (%%x) : %x %x\n", 15, 0x15); // Affiche :  
// Affichage d'un entier exprimé en hexadécimal (%x) : f 15
```

💡 **%%d** permet d'afficher **%d** au lieu de le remplacer par une valeur.

Mémoire

- Chaque variable occupe 1 ou plusieurs mots mémoires (octets).
- Le type de la variable détermine sa taille en mots mémoires (octets).
Un **int** occupe 4 octets, un **char** occupe 1 seul octet...
- Sans bidouillage, un mot mémoire ne peut pas être partagé par plusieurs variables.
Ainsi, pour représenter un simple booléen (1 bit), on est obligé d'utiliser un mot mémoire entier (1 octet : plus petite unité adressable en mémoire).
- L'adresse d'une variable occupant plusieurs mots mémoires est celle de son premier mot mémoire.
- Les mots mémoire d'une même variable sont contigus dans la mémoire.
On ne peut pas avoir le début à l'adresse **0x4a68**, d'autres variables au milieu, puis la suite à l'adresse **0xff08**.

Mémoire - Exemple d'un entier en mémoire centrale

. . 0110101110110101100100100010001111011011 . .

0xc5fd	0xc5fe	0xc5ff	0xc600	0xc601	0xc602
--------	--------	--------	--------	--------	--------

? | | -----Entier de 4 octets----- | ?

Chaque octet a sa propre adresse.

L'entier, stocké sur 4 octets, a pour adresse : **0xc5fd**

Mémoire - Exemple d'un entier en mémoire centrale

Adresse	Bits en mémoire								Valeur
....	...								...
0xef3c4a6 <u>8</u>	0	0	1	0	0	1	1	1	Quelque chose
0xef3c4a6 <u>9</u>	0	0	0	1	0	1	0	0	Quelque chose
0xef3c4a6<u>a</u>	0	0	0	0	0	0	0	0	Un entier - d'adresse : 0xef3c4a6<u>a</u> - de valeur : 11
0xef3c4a6 <u>b</u>	0	0	0	0	0	0	0	0	
0xef3c4a6 <u>c</u>	0	0	0	0	0	0	0	0	
0xef3c4a6 <u>d</u>	0	0	0	0	1	0	1	1	
0xef3c4a6 <u>e</u>	0	0	0	0	0	0	0	0	Quelque chose
...	...								...

Chaque mot mémoire (ici d'un octet) possède une adresse. Un entier occupe généralement 4 octets.
Il occupera donc ici 4 mots mémoires (donc 4 octets).

Adresses

On obtient l'**adresse** d'une variable avec l'opérateur unaire¹ "&" (esperluette).

```
int a = 11;
```

```
printf("%d", a); // affiche la valeur de a
```

```
printf("%x", &a); // affiche l'adresse de a (&a) au format hexadécimal
```

¹unaire : qui n'a qu'un seul opérande

Adresses - Pourquoi les utiliser ?

Nous avons déjà utilisé l'esperluette — et donc les adresses — avec **scanf()**.

À votre avis, pourquoi la fonction scanf a-t-elle besoin de l'adresse de la variable du (sous-)programme qui l'appelle ?

```
int main() {  
    int a = 0;  
    scanf("%d", &a); // en appelant scanf, la fonction main  
}                  // lui fournit l'adresse de sa variable a
```

Adresses - Pourquoi les utiliser ?

- Imaginons que l'on crée une version simplifiée de **scanf** :
Sans utiliser d'adresse.

```
void scanfMaison(/* ... */, int variableAModifier) { // 2 On reçoit une copie de la
                                                    // valeur 0 dans variableAModifier
    // code compliqué
    variableAModifier = 5; // 3 On suppose que l'utilisateur a tapé 5, la copie est modifiée
}

int main(void) {
    int a = 0; // 1 a vaut 0
    scanfMaison("%d", a);
    printf("%d", a); // 4 a vaut toujours 0 (c'est une copie qui a pris la valeur 5), affiche 0
    return 0;
}
```

Adresses - Pourquoi les utiliser ?

Dans le main, pourquoi **a** ne vaut pas **5** après l'appel de **scanfMaison** ?

- Car **a** est **passé par valeur**.

On copie sa valeur dans la variable de la fonction appelée (**scanfMaison**).

- Le sous-programme **scanfMaison** travaille donc sur une **copie de la valeur** de la variable **a** du main (une copie de la valeur **0**, qui s'appelle **variableAModifier** à l'intérieur de **scanfMaison**).

Adresses - Pourquoi les utiliser ?

Pour que la procédure **scanfMaison** puisse modifier directement la valeur de la variable **a** du **main**, ce dernier ne va non plus lui transmettre une copie de la valeur de sa variable **a** (0) mais une **copie de son adresse en mémoire** (ex : **0x3ba1**). Ainsi, la procédure **scanfMaison** pourra modifier directement l'espace mémoire de la variable **a** du **main** :

```
void scanfMaison(/* ... */ , int* adrVarAModifier) { // 3 adrVarAModifier reçoit comme valeur l'adresse 0x3ba1
    // code compliqué
    *variableAModifier = 5; // 4 On suppose que l'utilisateur a tapé 5,
                           // on écrit la valeur 5 à l'adresse contenue dans adrVarAModifier (en 0x3ba1)
                           // (l'adresse de la variable a du main)
}

int main(void) {
    int a = 0;           // 1 a vaut 0, on suppose qu'elle est à l'adresse 0x3ba1
    scanfMaison("%d", &a); // 2 Nous passons 0x3ba1 en paramètre
    printf("%d", a);      // 5 a vaut maintenant 5 🤖 le printf affiche 5
    return 0;
}
```

Adresses - Pourquoi les utiliser ?

Revenons à la fonction **scanf**.

Le **main** lui fournit bien l'adresse de sa variable **a** (et non pas sa valeur), afin que le **scanf** puisse modifier directement sa valeur.

```
scanf("%d", &a); // ici, %d indique à scanf que l'on souhaite écrire un  
                // entier (4 octets), à partir de l'octet d'adresse &a
```

Sans ce mécanisme, une fonction ne peut modifier que ses propres variables (celles déclarées dans son **bloc** *(ou de façon globale (pas bien))*).

Adresses - Pourquoi les utiliser ?

Et cette étoile (*) à côté du **int**, c'est quoi ?

```
void scanfMaison(/* ... */, int* adrVarAModifier) { // 3 adrVarAModifier reçoit comme valeur l'adresse 0x3ba1
    // code compliqué
    *variableAModifier = 5; // 4 On suppose que l'utilisateur a tapé 5,
                           // on écrit la valeur 5 à l'adresse contenue dans adrVarAModifier (en 0x3ba1)
                           // (l'adresse de la variable a du main)
}

int main(void) {
    int a = 0;           // 1 a vaut 0, on suppose qu'elle est à l'adresse 0x3ba1
    scanfMaison("%d", &a); // 2 Nous passons 0x3ba1 en paramètre
    printf("%d", a);      // 5 a vaut maintenant 5 🍷 le printf affiche 5
    return 0;
}
```

C'est un **pointeur**.

Pointeurs

Définition :

Un pointeur est une variable qui stocke une adresse mémoire.

La valeur d'un pointeur est une adresse. Un pointeur dispose comme toute variable de sa propre adresse.

```
int n = 10;    // n est un entier (un int), sa valeur est un entier : 10,  
              // son adresse mémoire est – par exemple – 0x9da3 (elle occupe 4 octets à partir de cette adresse).  
  
int* pN = &n; // pN est un pointeur sur un entier (un int*), sa valeur est l'adresse d'un entier (l'adresse de n) : 0x9da3,  
              // son adresse mémoire est – par exemple – 0xa58c (chaque variable a sa propre adresse).
```

Vocabulaire :

On dit qu'un pointeur pointe sur X s'il a pour valeur l'adresse de X . Ici, pN pointe sur n.

Pointeurs

Pour qu'une variable ne stocke non plus une valeur, mais une adresse, on ajoute un **astérisque (*)** à droite du type de ce dont on veut stocker l'adresse. C'est alors un **pointeur**.

	Type	Valeur/Contenu
<i>Pointeur sur un entier</i> (sur un <code>int</code>)	<code>int*</code>	Adresse d'un <code>int</code>
<i>Pointeur sur un nombre décimal</i> (sur un <code>float</code>)	<code>float*</code>	Adresse d'un <code>float</code>
<i>Pointeur sur un pointeur sur un entier</i> (sur un <code>int*</code>)	<code>int**</code>	Adresse d'un <code>int*</code>

NB : Le type d'un pointeur a une astérisque de plus que le type de la variable sur laquelle il pointe.

Pointeurs - Déclaration

La position de l'astérisque n'a pas d'importance :

```
int* pMonPointeur; // (conseillé)
// --- OU ---
int *pMonPointeur; // (très répandu)
// --- OU ---
int * pMonPointeur; // (rare)
// --- OU ---
int*pMonPointeur; // (rare)
```

Une bonne pratique consiste à nommer les pointeurs en commençant par un "p" (notation hongroise).



ATTENTION : il faut toujours initialiser ses variables, surtout les pointeurs !

Pointeurs - Initialisation

😌 Pensez à toujours initialiser vos variables, surtout les pointeurs.

Un pointeur peut ne pointer sur rien. On lui donne alors la valeur **NULL**.

```
int a = 11;  
int* pMonPointeur = &a; // pointeur qui pointe sur a  
  
// ou  
  
int* pMonPointeur = NULL; // pointeur qui ne pointe encore sur rien (0x0)  
  
if (pMonPointeur != NULL) { ... } // il faut alors tester si notre  
                                   // pointeur pointe sur quelque chose  
                                   // avant de l'utiliser
```

Pointeurs

Au cours du programme, un pointeur peut pointer sur plusieurs variables.

Sauf s'il est constant (`int* const pInt`).

⚠ Un `int*` ne peut pointer que sur un `int`.

```
int a = 5, b = 10;
```

```
int* pInt = NULL; // pInt ne pointe sur rien
```

```
if (pInt == NULL) { // exemple de test (facultatif)
    pInt = &a; // pInt pointe maintenant sur a
}
```

```
pInt = &b; // pInt pointe maintenant sur b
```

Pointeurs - Accès à la valeur à l'adresse pointée

On accède à la valeur se trouvant à l'adresse **pointée** avec l'opérateur unaire `*` :

```
int entier = 5;
```

```
int* pEntier = &entier; // pEntier pointe sur entier
```

```
printf("%x\n", pEntier); // affiche la valeur de pEntier, donc l'adresse d'entier
```

```
printf("%d\n", *pEntier); // affiche 5 (la valeur pointée par pEntier)
```

```
printf("%x\n", &pEntier); // affiche l'adresse du pointeur pEntier (&pEntier)
```



ATTENTION : ne pas confondre avec la déclaration du pointeur (`int* pEntier;`), ni avec la multiplication (`a*b`).

Opérateur * - Trois utilisations à distinguer

- Pour déclarer un pointeur :

int* p = &a; // On crée un pointeur p qui pointe sur l'entier a

- Pour accéder à la valeur pointée :

***p** = 5; // a vaut maintenant 5

- Pour multiplier :

3p** // Donne comme résultat 15

Pointeurs - Exemple

Comment accéder à la valeur/l'adresse ?

```
#include <stdio.h>
```

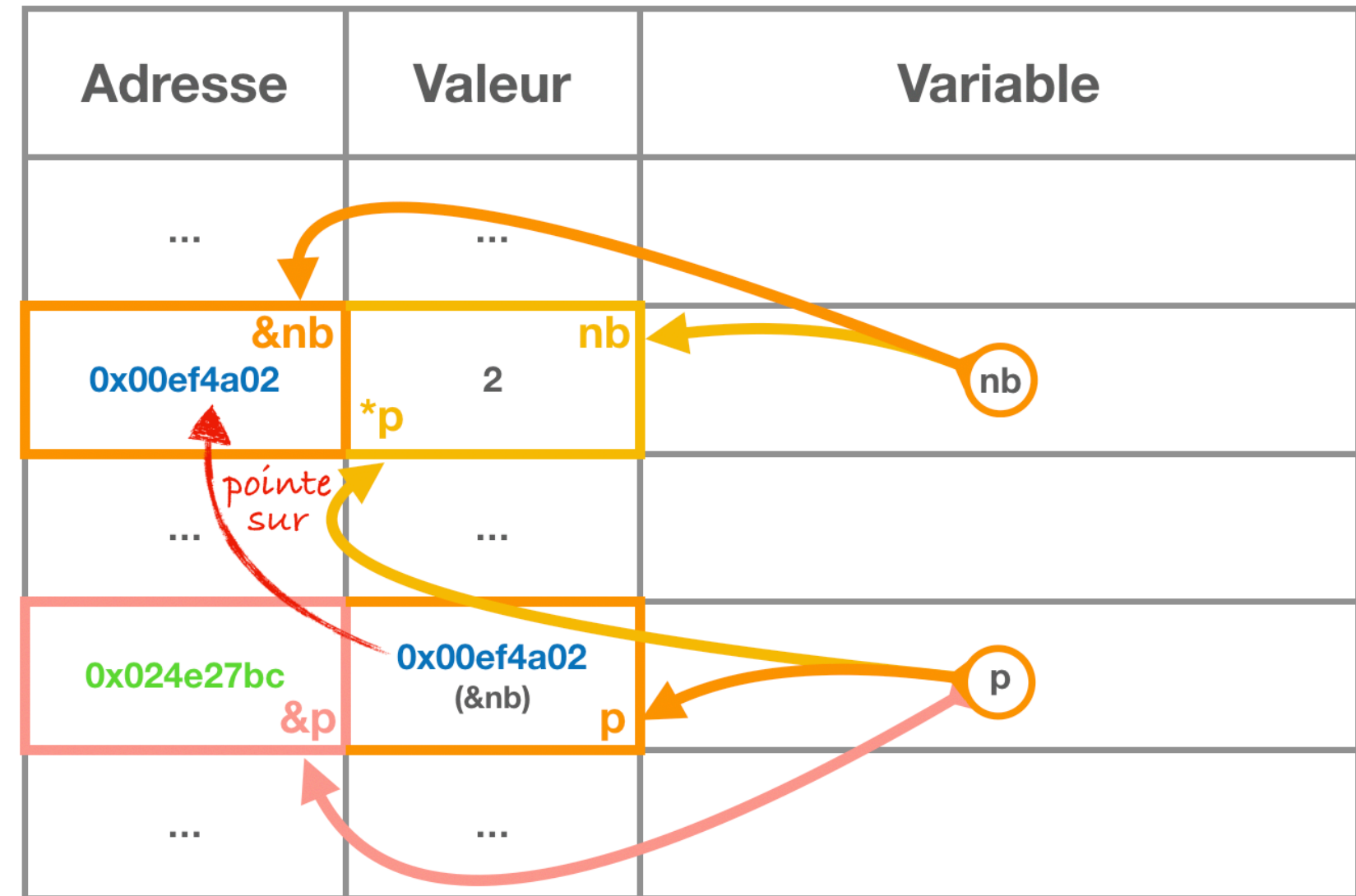
```
int main(void) {
```

```
    int nb = 2;
```

```
    int* p = &nb;
```

```
    return 0;
```

```
}
```



Pointeurs sur pointeurs sur ...

```
int entier = 5;
int* pEntier = &entier;    // pointeur sur un entier
int** ppEntier = &pEntier; // pointeur sur un pointeur sur un entier

printf("%x\n", pEntier);    // affiche l'adresse de entier

printf("%d\n", *pEntier);   // affiche 5 (la valeur à l'adresse pointée)

printf("%x\n", ppEntier);   // affiche l'adresse de pEntier

printf("%x\n", *ppEntier);  // affiche l'adresse de entier
                          // (la valeur de pEntier)

printf("%d\n", **ppEntier); // affiche 5
```

Compiqué... On reviendra dessus avec les listes chaînées ! Pour bien comprendre, **faites des schémas**.

Pointeurs sur structure

La notation **p->x** est équivalente à **(*p) . x** (utilisées pour les structures).

```
#include <stdio.h>
typedef struct {
    int x, y;
} MaStructure;
int main(void) {
    MaStructure s = {0,0};
    MaStructure* p = &s; // pointeur sur la structure s

1   s.x = 1;           // s.x vaut maintenant 1
    printf("%d\n", s.x);

2   /***** est équivalent à : *****/
    (*p).x = 2;        // s.x vaut maintenant 2 ((*p) accède à la structure pointée par p : s)
    printf("%d\n", (*p).x);

3   /***** est également équivalent à : *****/
    p->x = 3;           // s.x vaut maintenant 3 (p->x est équivalent à (*p).x)
    printf("%d\n", p->x);
    return 0;
}
```

Pointeurs - Résumé

- La valeur d'un pointeur est une adresse.
- Si **p** pointe sur **x**, alors la valeur de **p** correspond à l'adresse de **x** (**&x**).
- Si **p** pointe sur **x**, alors toute modification de ***p** modifie **x** et toute modification de **x** modifie ***p** puisque ***p** et **x** correspondent au même espace mémoire.

Pointeurs - Récapitulatif sur un court programme

```
int nb = 5;  
int* p = &nb;
```

	Valeur	Exemple(s) d'utilisation
nb	La valeur de la variable nb : 5 .	nb = 5; printf("%d", nb);
&nb	L'adresse en mémoire de la variable nb .	int* p = &nb; scanf("%d", &nb);
p	La valeur du pointeur p , donc l'adresse d'un entier, celle de l'entier nb (car p = &nb).	p = &nb;
*p	La valeur en mémoire là où pointe p , donc 5 .	*p = 6; (nb vaut maintenant 6) scanf("%d", p); printf("%d", *p);
&p	L'adresse du pointeur p .	int** pp = &p; (pointeur sur un pointeur)

Pointeurs - Remarque

```
int n = 0;
int pN = &n;
scanf("%d", &n);
// est équivalent à :
scanf("%d", pN); // ⚠ pas d'esperluette car contient l'adresse souhaitée
// est équivalent à :
scanf("%d", &*pN);
// est équivalent à :
scanf("%d", &*(&*(&*(&*(&*(&*(&*(&*(&*(&*pN)))))));
// ...
```

Les combinaisons `&*` et `*&` s'annulent ! Et ça semble logique, l'un accède à l'adresse de la valeur, et l'autre à la valeur de l'adresse.

Passage par valeur/adresse

- **Passage par valeur** : on envoie une copie de la valeur à la fonction qu'on appelle.
- **Passage par adresse** : on envoie une copie de l'adresse à la fonction qu'on appelle.

Passage par valeur/adresse

```
int birthdayYear = 0;  
scanf("%d", &birthdayYear);  
printf("Vous êtes né en %d.", birthdayYear);
```

- **printf** ne fait que lire la valeur des variables qui lui sont passées en paramètres. Le passage par valeur est donc amplement suffisant. **printf** n'a pas à modifier les valeurs qu'on lui passe en paramètres. Il travaillera sur des copies des valeurs.
- **scanf**, en revanche, doit pouvoir modifier la valeur des variables qui lui sont passées en paramètre par la fonction appelante. C'est pourquoi on effectue ici un **passage par adresse**. Il travaillera donc directement sur le même espace mémoire que les variables de la fonction appelante.

Passage par valeur

On travaille sur une copie de la valeur

```
#include <stdio.h>
void uneProcedure(int a) { // passage par valeur : cette variable a récupère une copie de la valeur 5
    printf("Valeur : %d, adresse : %x\n", a, &a); // Valeur : 5, adresse : 0xecf86a68
}
int main(void) {
    int a = 5;
    printf("Valeur : %d, adresse : %x\n", a, &a); // Valeur : 5, adresse : 0xf5a32a4c
    uneProcedure(a);
    return 0;
}
```

La valeur est bien la même, mais l'espace mémoire est différent (on travaille sur une copie de la valeur). Ainsi, si on modifie **a** dans la procédure, la valeur de **a** dans le **main** (la fonction appelante) restera inchangée.

Passage par valeur - Avantages/inconvénients

Avantage :

- Permet d'éviter une modification accidentelle de valeur(s) qui se répercuterait en dehors de la fonction/procédure.

Inconvénients :

- Si aucun paramètre n'est passé par adresse, une fonction ne peut renvoyer qu'un seul résultat à sa fonction appelante (via le **return**).
- Peut être lourd car entraîne la copie de l'intégralité de la valeur.
ex : si on passe une structure de 600 octets en paramètres. Dans ce cas on peut la passer par adresse avec **const** pour éviter la modification.

Passage par adresse

On travaille sur une copie de l'adresse

Le passage par adresse consiste à transmettre à la fonction/procédure non pas une copie de la valeur, mais une copie de l'adresse où la valeur d'origine est stockée, afin notamment de pouvoir la modifier directement depuis cette fonction/procédure.

Passage par adresse - Exemple

```
#include <stdio.h>
// cette procédure prend en paramètre un pointeur sur un entier (donc une adresse, puisque
// la valeur d'un pointeur est une adresse)
void initEntier(int* pEntier) {
    *pEntier = 0; // on modifie la valeur (int) à l'adresse qui nous est transmise
                  // (l'étoile permet d'accéder à l'espace mémoire pointé (à la valeur)
}

int main(void) {
    int a; // entier non initialisé... (pas bien)

    // VERSION 1 (avec un pointeur dont la valeur est bien une adresse) – Rarement utilisée
    1 int* pA = &a;
      initEntier(pA); // on passe la valeur d'un pointeur sur entier (pA), donc l'adresse d'un entier, qui sera
                     // copiée dans le pointeur pEntier de la procédure initEntier

    // VERSION 2 (directement avec l'adresse, sans passer par un pointeur) – Souvent utilisée
    2 initEntier(&a); // on passe l'adresse de a qui sera copiée dans le pointeur pEntier de la procédure initEntier

    return 0;
}
```

Passage par adresse - Exemple

Même programme en passage par valeur

```
#include <stdio.h>
```

```
int initEntier(int entier) { // On reçoit une valeur que l'on stocke dans entier (potentiellement 654234)
    entier = 0; // on donne la valeur 0 à la copie locale
    return entier; // on retourne 0
}
```

```
int main(void) {
    int a; // entier non initialisé...
    a = initEntier(a); // on passe la valeur de a, qui est aléatoire car non initialisée (ex : 654234)
    return 0;
}
```

On remarque rapidement que **initEntier** n'a plus **aucun sens** en passage par valeur. Autant écrire **a = 0;** dans le **main**.

Comment pourrait-on créer une fonction/procédure équivalente qui n'initialise non pas un entier, mais trois ?

Comment pourrait-on créer une fonction/procédure équivalente qui n'initialise non pas un entier, mais trois ?

```
#include <stdio.h>
```

```
void initEntiers(int* pI1, int* pI2, int* pI3) {  
    *pI1 = 0;  
    *pI2 = 0;  
    *pI3 = 0;  
}
```

```
int main(void) {  
    int a, b, c;  
    initEntiers(&a, &b, &c);  
    return 0;  
}
```

💭 Cela serait-il possible en passage par valeur, pourquoi ?

Comment pourrait-on créer une fonction/procédure équivalente qui n'initialise non pas un entier, mais trois ?

```
#include <stdio.h>
```

```
void initEntiers(int* pI1, int* pI2, int* pI3) {  
    *pI1 = 0;  
    *pI2 = 0;  
    *pI3 = 0;  
}
```

```
int main(void) {  
    int a, b, c;  
    initEntiers(&a, &b, &c);  
    return 0;  
}
```

💭 Cela serait-il possible en passage par valeur, pourquoi ?

Non, car une fonction ne peut retourner qu'une seule valeur à sa fonction appelante (**return**).

Passage par adresse - Avantages/inconvénients

Avantages

- Permet de modifier directement les espaces mémoires du programme appelant (ses variables).
- Permet de transmettre plusieurs valeurs au programme appelant (contre une seule avec le passage par valeur avec un **return**), en les modifiant directement en mémoire.
- Permet de copier moins de mémoire : il vaut mieux copier l'adresse d'une structure lourde que la totalité de ses données.

Inconvénient

- Si on modifie la valeur d'un paramètre passé par adresse dans la fonction/procédure, il ne faut pas oublier qu'elle sera également modifiée dans la fonction appelante, voire ailleurs dans le programme.
- 💡 On peut utiliser le mot-clé **const** pour empêcher la modification.

Trace mémoire

Adresse	Valeur	Variable
---------	--------	----------

```
#include <stdio.h>

void initEntiers(int* pI1, int* pI2, int* pI3) {
    *pI1 = 0;
    *pI2 = 0;
    *pI3 = 0;
}

int main(void) {
    int a, b, c; // danger : variables non initialisées
    initEntiers(&a, &b, &c);
    return 0;
}
```

Lorsqu'on lance un programme, c'est la fonction `main()` qui est exécutée. Il s'agit du point de départ. Il n'y a au début aucune variable.

Trace mémoire

```
#include <stdio.h>

void initEntiers(int* pI1, int* pI2, int* pI3) {
    *pI1 = 0;
    *pI2 = 0;
    *pI3 = 0;
}

int main(void) {
    int a, b, c; // danger : variables non initialisées
    initEntiers(&a, &b, &c);
    return 0;
}
```

Adresse	Valeur	Variable
e8d11a80	?	a (int)
e926be42	?	b (int)
f32e0c24	?	c (int)

Chacun des 3 entiers a, b et c se voit allouer un espace mémoire de 4 octets (puisque ce sont des entiers).

Ils ont chacun une adresse mémoire (correspondant à l'adresse de leur premier mot mémoire).

Comme aucune valeur ne leur est affectée lors de leur déclaration (variables non initialisées), elles prennent la valeur correspondant aux bits déjà présents à cet espace mémoire.

Trace mémoire

```
#include <stdio.h>

void initEntiers(int* pI1, int* pI2, int* pI3) {
    *pI1 = 0;
    *pI2 = 0;
    *pI3 = 0;
}

int main(void) {
    int a, b, c; // danger : variables non initialisées
    initEntiers(&a, &b, &c);
    return 0;
}
```

Adresse	Valeur	Variable
&a e8d11a80	?	a (int)
&b e926be42	?	b (int)
&c f32e0c24	?	c (int)
e4c05a58	&a e8d11a80	pI1 (int*)
3d121b60	&b e926be42	pI2 (int*)
e8d11a48	&c f32e0c24	pI3 (int*)

La procédure initEntiers est appelée avec les adresses des entiers a, b, et c en paramètres.

Ces adresses sont alors affectées respectivement aux paramètres pI1, pI2 et pI3 de la procédure qui sont bien des pointeurs sur entier.

Trace mémoire

```
#include <stdio.h>

void initEntiers(int* pI1, int* pI2, int* pI3) {
    *pI1 = 0;
    *pI2 = 0;
    *pI3 = 0;
}

int main(void) {
    int a, b, c; // danger : variables non initialisées
    initEntiers(&a, &b, &c);
    return 0;
}
```

Adresse	Valeur	Variable
&a e8d11a80	*pI1 0 a	a (int)
e926be42	?	b (int)
f32e0c24	?	c (int)
e4c05a58	e8d11a80	pI1 (int*)
3d121b60	e926be42	pI2 (int*)
e8d11a48	f32e0c24	pI3 (int*)

Trace mémoire

```
#include <stdio.h>

void initEntiers(int* pI1, int* pI2, int* pI3) {
    *pI1 = 0;
    *pI2 = 0;
    *pI3 = 0;
}

int main(void) {
    int a, b, c; // danger : variables non initialisées
    initEntiers(&a, &b, &c);
    return 0;
}
```

Adresse	Valeur	Variable
e8d11a80	0	a (int)
e926be42	0	b (int)
f32e0c24	?	c (int)
e4c05a58	e8d11a80	pI1 (int*)
3d121b60	e926be42	pI2 (int*)
e8d11a48	f32e0c24	pI3 (int*)

Trace mémoire

```
#include <stdio.h>

void initEntiers(int* pI1, int* pI2, int* pI3) {
    *pI1 = 0;
    *pI2 = 0;
    *pI3 = 0;
}

int main(void) {
    int a, b, c; // danger : variables non initialisées
    initEntiers(&a, &b, &c);
    return 0;
}
```

Adresse	Valeur	Variable
e8d11a80	0	a (int)
e926be42	0	b (int)
f32e0c24	0	c (int)
e4c05a58	e8d11a80	pI1 (int*)
3d121b60	e926be42	pI2 (int*)
e8d11a48	f32e0c24	pI3 (int*)

Trace mémoire

```
#include <stdio.h>

void initEntiers(int* pI1, int* pI2, int* pI3) {
    *pI1 = 0;
    *pI2 = 0;
    *pI3 = 0;
}

int main(void) {
    int a, b, c; // danger : variables non initialisées
    initEntiers(&a, &b, &c);
    return 0;
}
```

Adresse	Valeur	Variable
e8d11a80	0	a (int)
e926be42	0	b (int)
f32e0c24	0	c (int)

Nous arrivons à une accolade fermante indiquant la fin de la procédure. Toutes les variables qui y ont été déclarées (paramètres et entre les accolades) sont libérées. Leur espace mémoire est à nouveau libre.

Trace mémoire

Adresse	Valeur	Variable
---------	--------	----------

```
#include <stdio.h>

void initEntiers(int* pI1, int* pI2, int* pI3) {
    *pI1 = 0;
    *pI2 = 0;
    *pI3 = 0;
}

int main(void) {
    int a, b, c; // danger : variables non initialisées
    initEntiers(&a, &b, &c);
    return 0;
}
```

Nous arrivons à la fin de la fonction `main`. Toutes les variables qui y ont été déclarées sont libérées et le programme se termine.

Passage par valeur/adresse - Exemple 1

Faisons la trace mémoire de ce programme.

```
#include <stdio.h>
```

```
void increment2Val(int *pA, int *pB, int increment) {  
    *pA += increment;
```

```
    // increment est passée par valeur, c'est-à-dire que si on modifie sa valeur ici,  
    // elle restera inchangée là où la procédure est appelée (dans le main) :  
    increment += 5; // on ne modifie que la copie dans la procédure
```

```
    *pB += increment; // la valeur d'incrément a bien changé
```

```
}
```

```
int main(void) {
```

```
    int a = 1, b = 1, inc = 1;
```

```
    increment2Val(&a, &b, inc);
```

```
    printf("[main] a = %d, b = %d, inc = %d\n", a, b, inc); // inc n'a pas changé : [main] a = 2, b = 7, inc = 1
```

```
    return 0;
```

```
}
```

Passage par valeur/adresse - Exemple 2

Faisons la trace mémoire de ce programme.

```
#include <stdio.h>

int incrementParValeur(int n, int i) {
    printf("[incrementParValeur] Adresse : %x, valeur : %d (on travaille sur une copie)\n", &n, n);
    return n + i; // On retourne la valeur telle qu'elle devrait être, sans la modifier directement
}

void incrementParAdresse(int *pN, int i) {
    printf("[incrementParAdresse] Adresse : %x, valeur : %d (on travaille sur le même espace mémoire)\n", pN, *pN);
    *pN += i; // On modifie directement la valeur à l'adresse que l'on nous donne
}

int main(void) {
    int n = 5;
    printf("[main] Adresse : %x, valeur : %d\n\n", &n, n);
    incrementParValeur(n, 2);
    printf("[main] Valeur après incrementParValeur (1/2) : %d (la valeur n'a pas été modifiée)\n\n", n);
    n = incrementParValeur(n, 2);
    printf("[main] Valeur après incrementParValeur (2/2) : %d (la valeur retournée a été affectée à n)\n\n", n);
    incrementParAdresse(&n, 2);
    printf("[main] Valeur après incrementParAdresse : %d (la valeur est directement modifiée en mémoire)\n", n);
    return 0;
}
```

Les tableaux - Passés par adresse

Comme vous le savez, les tableaux sont passés par adresse.

Si dans une fonction, on modifie un tableau reçu en paramètres, alors l'original (dans la fonction appelante) est modifié également. Car il s'agit du même tableau.

En réalité, un tableau est un pointeur. Un tableau ne stocke pas plusieurs valeurs mais l'adresse de la première case du tableau, et donc du tableau lui-même.

Rappel : toutes les cases d'un même tableau sont voisines dans la mémoire. Vous comprenez maintenant pourquoi !

Ainsi, si on a le tableau suivant : **int tab[10];**, alors :

tab[0] est équivalent à ***tab**

tab[n] est équivalent à ***(tab+n*sizeof(int))**